Le jeu des allumettes

Partie 1 : version console (brute)

Règle du jeu

Il y a **20 allumettes** sur la table. Le joueur et l'ordinateur jouent chacun leur tour, en commençant par le joueur. À chaque tour, on retire 1, 2 ou 3 allumettes. **Celui qui prend la dernière allumette a perdu.**

Travail demandé

Écris un programme Python qui :

- affiche le nombre d'allumettes restantes à chaque tour, à l'aide d'une chaîne de formatage ;
- demande au joueur combien il en prend, et redemande si son choix est interdit ;
- ne s'arrête pas si le joueur tape autre chose qu'un nombre (utilise `try` / `except`) ;
- fait jouer l'ordinateur avec la règle suivante : il prend `(allumettes - 1) % 4` allumettes, ou 1 si ce calcul donne 0 ;
- annonce le gagnant à la fin.

Question bonus

L'ordinateur suit une stratégie. Combien d'allumettes le joueur doit-il prendre au premier tour pour être sûr de gagner ?

Le jeu des allumettes

Partie 1 : version console (guidée)

Règle du jeu

Il y a **20 allumettes** sur la table. Le joueur et l'ordinateur jouent chacun leur tour, en commençant par le joueur. À chaque tour, on retire 1, 2 ou 3 allumettes. **Celui qui prend la dernière allumette a perdu.**

Étape 1 : les variables

Crée une variable `allumettes` qui vaut 20, et une variable `tour_joueur` qui vaut `True` (elle indique si c'est au joueur de jouer).

Étape 2 : la boucle de jeu

Le jeu continue tant qu'il reste des allumettes. Quelle boucle utiliser ? Avec quelle condition ? Au début de chaque tour, affiche le nombre d'allumettes restantes avec une chaîne de formatage, par exemple `f"Il reste {allumettes} allumettes"`.

Étape 3 : le tour du joueur

Si c'est au joueur de jouer, demande-lui un nombre avec `int(input(...))`. Ce nombre est interdit s'il est plus petit que 1, plus grand que 3, ou plus grand que le nombre d'allumettes restantes. Tant que la saisie n'est pas correcte, redemande.

Étape 4 : les erreurs de saisie avec try / except

Le problème. Si le joueur tape « trois » au lieu de 3, la conversion `int()` échoue. Python provoque alors une erreur de type `ValueError` et le programme s'arrête net :

```
Combien en prends-tu (1 à 3) ? trois
Traceback (most recent call last):
  ...
ValueError: invalid literal for int() with base 10: 'trois'
```

La solution. On place les instructions qui risquent de provoquer une erreur dans un bloc `try` (« essaie »). Python exécute ce bloc normalement. Si une erreur `ValueError` se produit, il abandonne immédiatement la suite du bloc `try` et saute dans le bloc `except ValueError`. Le programme ne plante plus : il continue après le `except`.

Exemple.

```
try:
    age = int(input("Ton âge ? "))
    print(f"Tu as {age} ans")
except ValueError:
    print("Ce n'est pas un nombre !")
```

Saisie	Ce qui se passe
L'utilisateur tape 15	`int()` réussit, le `print` du `try` affiche « Tu as 15 ans ». Le bloc `except` est ignoré.
L'utilisateur tape quinze	`int()` échoue : le `print` du `try` est sauté, et Python affiche « Ce n'est pas un nombre ! ».

Redemander jusqu'à ce que ce soit bon. Pour reposer la question en cas d'erreur, on met le `try` dans une boucle `while` contrôlée par une variable `ok` :

```
ok = False
while not ok:
    try:
        age = int(input("Ton âge ? "))
        ok = True
    except ValueError:
        print("Ce n'est pas un nombre, recommence !")
```

La ligne `ok = True` est placée juste après la conversion. Si la conversion échoue, Python saute directement dans le `except` : `ok` reste à `False` et la boucle recommence. Si elle réussit, `ok` passe à `True` et la boucle s'arrête.

Dans le jeu des allumettes, il y a deux raisons de redemander : le joueur n'a pas tapé un nombre (c'est le rôle du `except`), ou il a tapé un nombre interdit (c'est le rôle d'un `if` placé dans le `try`). La variable `ok` ne doit passer à `True` que si le nombre est valide **et** autorisé. Pense à afficher un message différent dans chaque cas.

Étape 5 : le tour de l'ordinateur

Sinon, c'est à l'ordinateur. Il prend `(allumettes - 1) % 4` allumettes. Si ce calcul donne 0, il en prend 1. Affiche combien il en prend.

Étape 6 : fin du tour

Retire les allumettes prises du tas, puis change de joueur. Astuce : `tour_joueur = not tour_joueur` inverse `True` et `False`.

Étape 7 : le gagnant

Après la boucle, le dernier à avoir joué a pris la dernière allumette, donc il a perdu. Mais attention, `tour_joueur` a déjà été inversé ! Si `tour_joueur` vaut `True`, qui a joué en dernier ? Affiche un message pour annoncer le résultat.

Squelette à compléter

Remplace chaque `...` par le bon code.

```
allumettes = 20
tour_joueur = True

while ... :
    ...
    if ... :
        ok = False
        while ... :
            try:
                prise = int(input("Combien en prends-tu (1 à 3) ? "))
                if ... :
                    ok = True
                else:
                    ...
            except ValueError:
                ...
    else:
        prise = ...
        if ... :
            prise = 1
        ...
    allumettes = ...
    tour_joueur = ...

if ... :
    ...
else:
    ...
```

Question bonus

L'ordinateur suit une stratégie. Combien d'allumettes le joueur doit-il prendre au premier tour pour être sûr de gagner ?

Le jeu des allumettes

Partie 2 : version graphique (brute)

Règle du jeu

Les règles sont les mêmes que dans la partie 1. L'ordinateur prend `(allumettes - 1) % 4` allumettes, ou 1 si ce calcul donne 0.

Travail demandé

Avec la bibliothèque `nsi_ui`, crée une interface graphique pour jouer au jeu des allumettes. Elle doit contenir :

- un texte qui affiche le nombre d'allumettes restantes ;
- trois boutons « Prendre 1 », « Prendre 2 » et « Prendre 3 », placés côte à côte ;
- un texte qui affiche les messages : ce que l'ordinateur a joué, un coup interdit, le gagnant.

Quand le joueur clique sur un bouton, le programme retire ses allumettes, puis l'ordinateur joue immédiatement. Une fois la partie terminée, les boutons ne doivent plus rien faire.

Le fichier `nsi_ui.py` doit se trouver dans le même dossier que ton programme.

Pour aller plus loin

- Ajoute un bouton « Rejouer » qui remet la partie à zéro.
- Utilise `set_background` pour colorer le message en vert quand le joueur gagne et en rouge quand il perd.

Le jeu des allumettes

Partie 2 : version graphique (guidée)

Règle du jeu

Les règles sont les mêmes que dans la partie 1. L'ordinateur prend `(allumettes - 1) % 4` allumettes, ou 1 si ce calcul donne 0.

Ce qui change

En version graphique, il n'y a plus de boucle `while` pour faire tourner le jeu : c'est la programmation **réactive**. Le programme attend que le joueur clique sur un bouton, et c'est ce clic qui déclenche la fonction de rappel. Le tour de l'ordinateur se fait juste après, dans la même fonction.

Étape 1 : les variables

Crée une variable `allumettes` qui vaut 20, et une variable `fini` qui vaut `False` (elle indique si la partie est terminée).

Étape 2 : l'interface

Dans un conteneur vertical, crée un `label` nommé `reste`, puis un conteneur horizontal avec les trois boutons, puis un `label` nommé `message`. Termine par `main_loop()`.

Étape 3 : un bouton avec un paramètre

La fonction `button` accepte un troisième paramètre : `button("Prendre 2", jouer, 2)` appelle `jouer(2)` quand on clique. Les trois boutons utilisent donc la même fonction `jouer(prise)`.

Étape 4 : modifier une variable dans une fonction

Pour qu'une fonction puisse modifier une variable créée en dehors d'elle, on écrit `global allumettes, fini` sur la première ligne de la fonction. Sans cette ligne, Python provoque une erreur.

Étape 5 : les cas impossibles

Dans `jouer`, vérifie d'abord si la partie est finie, puis si le joueur prend plus d'allumettes qu'il n'en reste. Dans ces deux cas, affiche un message avec `set_text(message, ...)` et ne fais rien d'autre.

Étape 6 : le coup du joueur, puis celui de l'ordinateur

Sinon, retire les allumettes du joueur. S'il n'en reste plus, il a perdu : affiche-le et passe `fini` à `True`. Sinon, l'ordinateur joue avec sa stratégie, puis on vérifie de la même façon s'il a pris la dernière.

Étape 7 : mettre à jour l'affichage

À la fin de `jouer`, mets à jour le label `reste` avec une chaîne de formatage.

Squelette à compléter

Remplace chaque `...` par le bon code. Le fichier `nsi_ui.py` doit se trouver dans le même dossier.

```
from nsi_ui import *

allumettes = 20
fini = False

def jouer(prise):
    global allumettes, fini
    if ... :
        set_text(message, ...)
    elif ... :
        set_text(message, ...)
    else:
        allumettes = ...
        if ... :
            ...
            fini = ...
        else:
            ordi = ...
            if ... :
                ordi = 1
            allumettes = ...
            if ... :
                ...
                ...
            else:
                set_text(message, f"...")
    set_text(reste, f"...")

begin_vertical()
reste = label("Il reste 20 allumettes")
begin_horizontal()
button("Prendre 1", jouer, 1)
...
...
end_horizontal()
message = label("À toi de jouer !")
end_vertical()
main_loop()
```

Pour aller plus loin

- Ajoute un bouton « Rejouer » qui appelle une fonction `rejouer()` : elle remet `allumettes` à 20, `fini` à `False` et met à jour les deux labels (pense à `global` !).
- Utilise `set_background(message, "green")` quand le joueur gagne et `"red"` quand il perd.

Le jeu des allumettes

Partie 3 : variante avec champ de saisie et try / except

Consigne

Reprends ta version graphique de la partie 2. Remplace les trois boutons « Prendre 1/2/3 » par un **champ de saisie** où le joueur tape le nombre d'allumettes qu'il prend, et un bouton « Jouer ». Le programme ne doit jamais planter, même si le joueur tape n'importe quoi.

Étape 1 : d'où vient l'erreur ?

La fonction `get_int(champ)` lit le texte tapé dans le champ et le convertit en entier avec `int()`. Si le joueur tape « trois », la conversion échoue et provoque une erreur `ValueError`, exactement comme `int(input())` dans la partie 1.

Étape 2 : une nouvelle fonction de rappel

Le bouton « Jouer » appelle une fonction `clic_jouer()`, sans paramètre. Elle lit la valeur du champ dans un bloc `try`, vérifie qu'elle est entre 1 et 3, puis appelle `jouer(prise)`. La fonction `jouer` de la partie 2 ne change pas : elle vérifie déjà s'il reste assez d'allumettes.

Étape 3 : afficher les erreurs

Si ce n'est pas un nombre, le bloc `except ValueError` affiche un message dans le label `message`. Si c'est un nombre hors de 1 à 3, un `if` affiche un autre message.

Étape 4 : pourquoi plus de boucle while ?

Dans la partie 1, il fallait une boucle `while not ok` pour redemander la saisie. Ici, c'est inutile : on affiche simplement l'erreur, et le programme attend le prochain clic. Le joueur corrige sa saisie et reclique. C'est tout l'intérêt de la programmation **réactive**.

Squelette à compléter

Garde les variables et la fonction `jouer` de la partie 2, puis remplace l'interface par ce code.

```
def clic_jouer():
    try:
        prise = ...
        if ... :
            set_text(message, ...)
        else:
            ...
    except ValueError:
        ...

begin_vertical()
reste = label("Il reste 20 allumettes")
champ = entry("Combien en prends-tu (1 à 3) ?")
button("Jouer", ...)
message = label("À toi de jouer !")
end_vertical()
main_loop()
```

Pour aller plus loin

- Après chaque coup, vide le champ de saisie avec `set_text(champ, "")`.
- Fais en sorte que la touche Entrée joue aussi : `entry` accepte une fonction en deuxième paramètre, appelée avec le texte tapé quand on appuie sur Entrée.